

Carrera:
Programación y WebMaster

Docente:
MTRO. Mario Alonso Segovia Gutiérrez

Materia:
Proyecto de una Plataforma Virtual

Alumno(s):
Salomon Campos Pablo Leonel

Fecha:
25 de Mayo 2026

Introducción

En esta actividad se analiza la plataforma Steam. Steam es una plataforma relacionada con la distribución digital de videojuegos, la gestión de bibliotecas, la comunidad de jugadores, pagos, actualizaciones, soporte y herramientas para desarrolladores. Por esa razón, resulta un buen ejemplo para estudiar cómo se aplicarían las etapas del SDLC en un proyecto grande y con muchos usuarios.



1. Descripción del sistema

Nombre del sistema: Steam.

- Problema que resuelve: Es la compra y administración de videojuegos para que el usuario no tenga que buscar instaladores en diferentes plataformas. Para los desarrolladores, ayuda a publicar productos.

a. Tipo de usuarios

- Jugadores
- Desarrolladores
- Soporte tecnico
- Administradores
- Comunidad

b. Funcionalidades principales

- Registro e inicio de sesión.
- Catálogo de videojuegos, búsquedas de videojuegos, recomendaciones.
- Carrito de compras, métodos de pago por regiones.
- Biblioteca personal para instalar, actualizar y iniciar juegos.
- Descarga de contenido de forma facil.
- Funciones sociales como amigos, chats, perfiles.
- Soporte técnico.

c. importancia del sistema

La importancia de la empresa es que para los jugadores mejora el acceso a videojuegos y poder actualizarlos. Para los desarrolladores de videojuegos, facilita el poder publicar en tienda digital.

2. Etapa de planeación

a. Objetivos del sistema

- Permitir que los usuarios compren y administren sus juegos de forma sencilla.
- Ofrecer una plataforma estable.
- Ofrecer una experiencia amplia y rápida al acceder a varios recursos.
- Facilitar la operación de desarrolladores mediante paneles y APIs.
- Servicios para interactuar con la comunidad para ofrecer mejor interaccion con los jugadores.

b. Alcance general

El alcance de la plataforma incluirea una aplicación de escritorio, sitio web, aplicación móvil, panel de administración, pagos, catálogos, bibliotecas y soporte tecnico.

c. Recursos necesarios

- Equipo humano.
- Infraestructura.
- Tecnologías
- Negocio

d. Riesgos principales

- Sobrecarga del sistema.
- Fraude y robo de cuentas.
- Errores en pagos.
- Problemas de seguridad.
- Mala experiencia de usuario.
- Problemas legales o regionales.

-

e. Tiempo estimado y costos aproximados

Para desarrollar una plataforma como Steam, el tiempo podría estimarse entre 20 a 40 meses, considerando el análisis del mercado, el diseño, desarrollo, pruebas, implementación y ajustes durante el desarrollo de la plataforma. El costo sería demasiado alto porque requiere infraestructura, seguridad, soporte. En una estimación se puede deducir en un costo de varios millones de dólares en una escala de entre 40 y 100 millones de dólares.

3. Análisis de requerimientos

a. Requerimientos funcionales

ID	Requerimiento funcional
RF-01	El sistema debe permitir el registro, inicio de sesión.
RF-02	El sistema debe mostrar un catálogo de juegos.
RF-03	El sistema debe permitir agregar juegos al carrito de compras.
RF-04	El sistema debe permitir descargar, instalar, actualizar y desinstalar videojuegos desde el cliente.
RF-05	El sistema debe permitir solicitar soporte y consultar el estado de tickets o solicitudes.

3.2 Requerimientos no funcionales

ID	Requerimiento no funcional
RNF-01	El sistema debe mantener alta disponibilidad durante todo el año.
RNF-02	Las operaciones de compra, inicio de sesión deben de ser rapidas.
RNF-03	La plataforma debe proteger datos personales.
RNF-04	El sistema debe ser escalable para soportar crecimiento de usuarios, juegos.
RNF-05	La interfaz debe ser clara, accesible y usable en escritorio, web y móvil.

4. Diseño del sistema

a. Arquitectura general

La arquitectura podría ser cliente-servidor con microservicios o módulos independientes. El usuario interactuaría con una aplicación de escritorio, una aplicación web o una app móvil. Estas interfaces consumirían APIs del backend, donde se manejarían servicios de autenticación, catálogo, pagos, biblioteca, descargas, comunidad, soporte y administración.

b. Base de datos

Entidad	Datos
Usuarios	ID, nombre, correo, contraseña, región.
Juegos	ID, título, desarrollador, precio, región, requisitos, descripción.
Compras	ID de compra, usuario, método de pago, moneda, impuestos, estado y fecha.
Biblioteca	Licencias, estado de instalación.
Comunidad	Reseñas, comentarios, reportes.
Soporte	Tickets, categorías, evidencias, estados, resolución.
Desarrolladores	Proyectos, versiones y permisos.

c. Tecnologías

- Frontend web: HTML, CSS, JavaScript, React.
- Backend: Java, Go, Node.js, Python o C# con APIs RESTL.
- Base de datos: PostgreSQL o MySQL para datos transaccionales.

d. Interfaces

- Pantalla de inicio de sesión y recuperación de cuenta.
- Página principal con ofertas y recomendaciones.
- Página de detalle de juego con precio, descripción, requisitos, reseñas e imágenes del juego.
- Carrito y proceso de compra.
- Biblioteca de juegos.
- Perfil de usuario.
- Panel de soporte tecnico.

e. Seguridad

- Cifrado de contraseñas.
- Autenticación multifactor para cuentas.
- Tokens de sesión con expiración y renovación segura.
- Monitoreo de intentos por direcciones Ip's a las acostumbradas a acceder.
- Permisos por rol para usuarios, desarrolladores, soporte y administradores.

f. APIs o servicios externos

- Pasarelas de pago y proveedores bancarios.
- Servicios de correo para verificación, compras y recuperación de cuenta.
- CDN para descargas rápidas de juegos y actualizaciones.
- APIs de nube para guardar los datos.
- Sistemas antifraude y validación regional.

5. Desarrollo**a. Roles del equipo**

- Project Manager
- Analista de requerimientos
- Diseñador UX/UI
- Desarrollador frontend
- Desarrollador Backend

b. Tecnologías de desarrollo

- Git para control de versiones y ramas de desarrollo.
- GitHub para repositorios.
- Docker para ambientes reproducibles.
- Frameworks backend y frontend.

c. Organización del trabajo

El trabajo podría organizarse usando la metodología ágil, dividiendo el proyecto en sprints en periodos cortos o largos. Por ejemplo en el primer sprint se desarrollaría la categoría de cuentas y catálogo de videojuegos, después compras y biblioteca; luego descargas, comunidad, soporte tecnico.

Al igual para tener un control de versiones usando git.

d. Posibles dificultades

- Integrar pagos internacionales y métodos locales.
- Mantener descargas estables cuando muchos usuarios actualizan o instalan juegos al mismo tiempo.
- Controlar fraudes, cuentas robadas.
- Sincronizar información del cliente si accede desde cualquier dispositivo.
- Mantener rendimiento alto en todo momento.
- Coordinar equipos grandes y módulos dependientes entre sí.

6. Pruebas

Tipo de pruebas	Ejemplso
Pruebas funcionales	Verificar registro, inicio de sesión, compra de juego, descargar, actualizaciones, solicitud de soporte.
Pruebas de seguridad	Probar autenticación multifactor, sesiones, permisos, cifrados.
Pruebas de rendimiento	Simular miles o millones de usuarios navegando ofertas, comprando, descargando videojuegos.
Pruebas de integración	Validar integración con las apps o Api's de pagos, correos, soporte, bases de datos y notificaciones.
Pruebas de usuario	Observar si los usuarios pueden encontrar un juego sin dificultad, comprarlo o instalarlo sin dificultades.

7. Implementación

a. Infraestructura y hosting

- Nube híbrida o infraestructura propia para servicios críticos.
- Balanceadores de carga para distribuir peticiones.
- Bases de datos con respaldo automatico cada 12 o 24h.
- Planes de recuperación ante perdidas de datos.

b. Publicación

Primero se publicaría en una beta cerrada donde ciertas personas pueden acceder a la plataforma y hacer testeos en donde se encuentren bugs menores o criticos, luego pasaria a una beta abierta en donde los usuarios probarian el sistema y conforme el uso del mismo se corroboran bugs menores y luego de ello pasaria a produccion ya con el producto terminado.

c. Capacitación de usuarios

Se crearán manuales, tutoriales, preguntas frecuentes (FAQ), documentación y guías de solución de problemas.

d. Migración de información

Para la migracion de informacion seria necesario migrar cuentas, compras, licencias, reseñas, datos de juegos, tickets de soporte. La migración debe probarse con copias de datos antes de usar información real. También se deben crear respaldos para evitar pérdida de datos.

8. Mantenimiento

- **Corrección de errores:** solucionar fallas reportadas por usuarios.
- **Actualizaciones:** mejorar cliente, web, APIs, bases de datos y servicios de backend.
- **Nuevas funcionalidades:** agregar filtros, recomendaciones, herramientas de comunidad, opciones de seguridad.
- **Seguridad:** parchear vulnerabilidades, reforzar autenticación, revisar permisos y bloquear actividad sospechosa.
- **Optimización:** reducir tiempos de carga, mejorar búsqueda, cachear información.
- **Compatibilidad:** adaptar la plataforma a nuevos dispositivos, sistemas operativos y tecnologías de hardware.

9. Análisis y reflexión

1- ¿Qué etapa del SDLC consideras más importante y por qué?

La etapa más importante es el análisis de requerimientos, porque ahí se entiende el contexto que va a llevar la plataforma. Si los requerimientos se definen mal se pueden implementar funcionalidades que no eran necesarias en el momento afectando el desarrollo de esta al igual afecta si se da un mal desarrollo dejar vulnerabilidades graves haciendo que la plataforma sea insegura.

2- ¿Qué riesgos existirían si se omite una etapa?

Si se omite una etapa aumenta la probabilidad de errores por ejemplo si se cancela el diseño el sistema podría quedar desorganizado y no responsivo.

3- ¿Por qué el mantenimiento es crítico en sistemas modernos?

Es importante porque los sistemas cambian todo el tiempo por que las tecnologías se actualizan y el negocio necesita adaptarse si el negocio no se adapta al cambio de tecnologías puede hacer que este este mas lento y al pasar del tiempo quedaría como algo obsoleto.

4- ¿Cómo ayuda el SDLC a mejorar calidad y organización?

Ayuda a mejorar la calidad porque divide el proyecto en etapas y permite documentar necesidades del sistema y también mejora la organización del equipo por que cada rol del equipo sabría cual sería su area de trabajo.